

Project Proposal Report: Preference-Based Social Event Recommendation Robot

Bhavesh Gandhi, Khizer Abdul Rahim, Kamran Ali, Rahul Bale

1 Introduction & Project Overview

This project focuses on developing a socially interactive robot within the qiBullet simulation environment capable of engaging users, recognizing their presence, and providing personalized social event recommendations.

By leveraging a Bayesian Network, the robot dynamically handles the inherent uncertainty of human preferences, mood, and environmental contexts to deliver highly accurate, probabilistic recommendations.

2 System Architecture

To ensure modularity and smooth execution within the qiBullet environment, the system is divided into four interconnected layers:

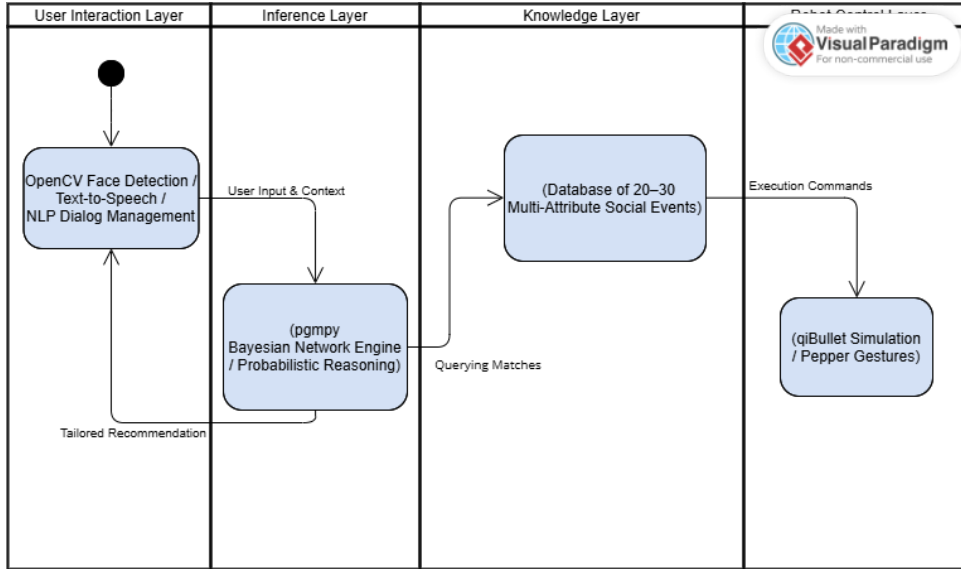


Figure 1: An Illustration of the System Architecture

3 Bayesian Network Design

The core intelligence of our project relies on a Directed Acyclic Graph (DAG) that maps user context and preferences directly to an Event Match Score and final Recommended Event.

This section describes the structure and probabilistic mechanics of the Bayesian Network designed for the event recommendation system. The model enables an interactive system or robot to interpret diverse, uncertain human state inputs and compute optimal event recommendations.

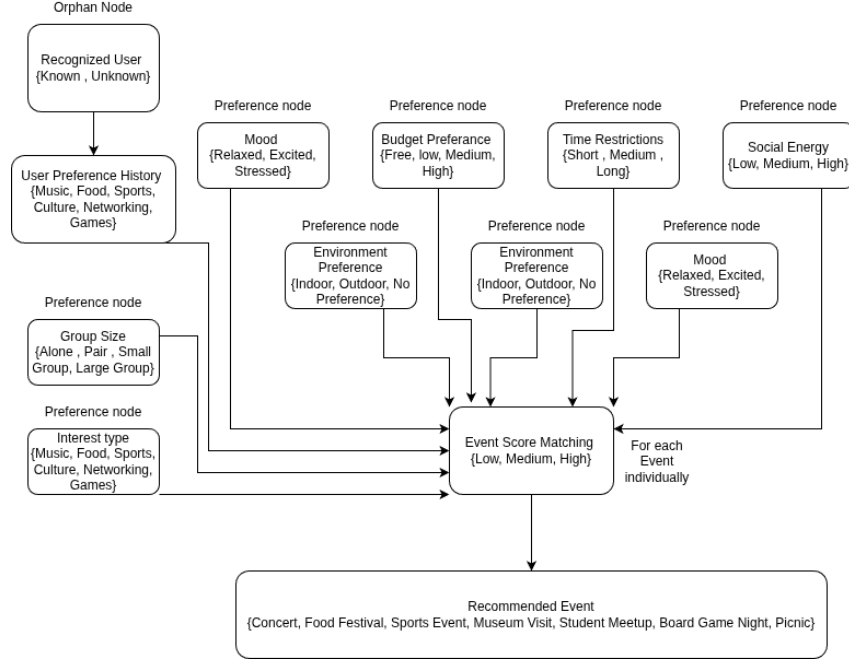


Figure 2: A sketch of the Bayesian Network Design

3.1 Inputs (Preference Nodes)

The parent nodes capture the user's current context, intrinsic preferences, and situational constraints. Each node possesses discrete states that feed directly into the matching processor:

- **Recognized User:** Binary states {Known, Unknown}. This serves as the causal parent to the user history node.
- **User Preference History:** Categorical states {Music, Food, Sports, Culture, Networking, Games}. It relies conditionally on whether the user is recognized by the system.
- **Group Size:** States {Alone, Pair, Small Group, Large Group} which represent social constraints.
- **Interest Type:** States {Music, Food, Sports, Culture, Networking, Games} specifying the explicit activity category desired.
- **Mood:** States {Relaxed, Excited, Stressed} capturing the user's emotional or psychological condition.
- **Budget Preference:** States {Free, Low, Medium, High} modeling financial boundaries.
- **Environment Preference:** States {Indoor, Outdoor, No Preference} detailing physical space constraints.
- **Time Restrictions:** States {Short, Medium, Long} defining temporal availability.
- **Social Energy:** States {Low, Medium, High} denoting capacity for interaction.

3.2 The Processing Step (Event Score Matching)

All conditional arrows from the preference nodes converge into a singular central node labeled **Event Score Matching**. This node functions as the primary decision-making and computational framework.

In a Bayesian Network, this node evaluates the joint conditional probability distribution given the observed values of all parent variables simultaneously. For any given event evaluated individually, it yields a posterior compatibility metric distributed across three discrete levels: {Low, Medium, High}.

3.3 The Output (Recommended Event)

The final directed edge links the score matching node to the terminal node labeled **Recommended Event**. Based on the calculated compatibility distributions, the system isolates the optimal event type from a discrete action selection set: {Concert, Food Festival, Sports Event, Museum Visit, Student Meetup, Board Game Night, Picnic}.

3.4 How the Math/Probabilities Work

For this network to function, every node contains a Conditional Probability Table (CPT).

Because *Event Score Matching* has 10 different arrows pointing directly into it, its CPT is going to be massive. It has to calculate the probability of a match being {Low, Medium, High} for every single possible combination of your inputs. For example:

$$P(\text{Match} = \text{High} \mid \text{Mood} = \text{Stressed}, \text{Budget} = \text{Free}, \text{Group Size} = \text{Alone}, \dots) \quad (1)$$

As the note in your diagram says (“For each Event individually”), the system will run this network for every event in your database (e.g., a rock concert, a quiet museum trip).

If you are Stressed, Alone, and want something Free, a Museum Visit will likely result in a High match score. That High match score will then drastically shift the probabilities in the final Recommended Event node to favor a Museum Visit over a loud Concert.

4 Technology Stack & Implementation

The project utilizes a robust, Python-centric stack optimized for robotics and probabilistic machine learning:

- **Simulation & Robotics:** qiBullet coupled with PyBullet to simulate a physical Pepper robot capable of performing multimodal greetings and physical gestures.
- **Computer Vision:** OpenCV for real-time webcam face detection, paired with the `face_recognition` library to distinguish between “Known” and “Unknown” users.
- **Bayesian Inference:** pgmpy (Python General Markovian Probability y-field) to construct nodes, define CPTs, and compute real-time preference updates.
- **Dialog & Safety:** A Python-based Finite State Machine (FSM) handles conversation flow cleanly, integrated with string-filtering regex layers to detect and gracefully filter out abusive language or inputs.